

CRC校验

一、CRC定义：

CRC (Cyclic Redundancy Check, 循环冗余校验) 是一种基于**多项式除法原理**的**检错算法**，用于检测数据块在传输或存储过程中是否发生了**位错误**（如**比特翻转**、**数据损坏**）。

它通过对原始数据附加一个固定长度的**校验码 (Checksum)**，接收方使用相同的算法重新计算并比对，从而判断数据是否出错。

二、CRC校验原理：

你有一段要发送的数据（比如一串数字或字节），你想确保这段数据在传输过程中没有发生错误（比如某个 0 变成 1，或 1 变成 0）。

于是你使用一种特殊的“算法”（基于多项式），对这段数据计算出一个短的“指纹”（即 CRC 校验码，通常是 2~4 字节），并把它附加在数据后一起发送。

接收方收到数据后，**用同样的算法对收到的数据重新计算 CRC 校验码**，然后与你发送的校验码进行比对：

- 如果一致 → 数据很可能没问题
- 如果不一致 → 数据很可能出错了

注意：CRC 只能检测错误，不能纠正错误！

三、CRC工作流程：

- 发送端**：有原始数据（比如一帧报文、一段字节流）。使用 **CRC 算法 (特定多项式)** 对这些数据计算出一个 **CRC 校验值 (Checksum, 通常是 16 位或 32 位)**。将 **原始数据 + CRC 校验码** 一起发送出去。
- 接收端**：收到数据后，**提取原始数据部分，用同样的 CRC 算法和多项式重新计算 CRC 值**。将计算得到的 CRC 值与接收到的 CRC 校验码进行比较：

一致 → 数据正确

不一致 → 数据可能损坏，请求重发或丢弃

四、CRC核心：多项式

CRC 的本质是基于**二进制多项式除法**，不同的 CRC 算法使用不同的**生成多项式 (Generator Polynomial)**，这决定了 CRC 的强度和校验码长度。

常见的 CRC 多项式有：

CRC类型	多项式(十六进制表示)	校验码长度	应用场景
CRC-8	0x07, 0x31, 0x9B 等	8bit	一些串口协议、嵌入式设备

CRC类型	多项式(十六进制表示)	校验码长度	应用场景
CRC-16	0x8005 (CRC-16-IBM) 、 0x1021 (CRC-16-CCITT)	16bit	Modbus (0x8005) 、CAN (15bit 类似) 、XMODEM、串口通信
CRC-16-Modbus	0x8005 (但 bit 取反)	16bit	Modbus 协议 (最常见!)
CRC-32	0xEDB88320 (IEEE 802.3 CRC32)	32bit	以太网、ZIP、PNG、文件校验、TCP/IP

在工业通信中，最常用的是：

- Modbus 使用的是 CRC-16 (多项式 0x8005，但计算后高低字节互换)
- CAN 使用的是 15 位 CRC (类似 CRC-16，但位数更短)
- 以太网、文件校验常用 CRC-32

五、以 Modbus 为例：CRC-16 校验

1. Modbus 使用的 CRC 类型：

- CRC-16-Modbus
- 多项式：0x8005 ($x^{16} + x^{15} + x^2 + 1$)
- 校验码长度：16 bit (2 字节)
- 计算后：CRC 低位在前，高位在后 (即字节顺序是 低字节在前 / Little Endian)

2. 计算方法 (简化描述)：

1. 初始化一个 CRC 寄存器为 0xFFFF
2. 对每一个数据字节：与 CRC 寄存器异或进行 8 次移位 + 判断 + 异或多项式 操作
3. 最后将 CRC 寄存器中的值 高低字节互换 (在 Modbus 中)
4. 得到最终的 2 字节 CRC 校验码，附加到数据帧末尾

注：实际开发中，不建议手动实现 CRC 算法，因为有大量现成的库、工具、代码可以直接调用

六、CRC校验码长度

常见的CRC校验码长度：

校验类型	校验码长度	说明
CRC-8	8bit (1字节)	校验能力较弱，用于简单场景
CRC-16	16bit (2字节)	最常用，如 Modbus、CAN、串口通信
CRC-32	32bit (4字节)	校验能力强，用于文件、网络、磁盘等

一般来说：

- CRC-16 能检测大部分常见的通信错误
- CRC-32 更强大，但占用空间也更大

七、CRC优缺点

优点	说明
检错能力强	能检测常见的数据传输错误，如位翻转、突发错误等
计算速度快	算法简单，适合嵌入式和实时系统
额外开销小	只需附加 1~4 字节的校验码
标准化程度高	多种现成算法和实现，广泛支持

缺点	说明
只能检错，不能纠错	发现错误后一般需要重传或丢弃
无法防止恶意篡改	CRC 不是加密算法，不具备安全性（需配合其它机制）
对特定错误可能漏检	虽然概率很低，但某些特定错误组合可能通过校验（相比更强的校验如 CRC32/Hash更易漏检）

八、CRC校验的应用场景

应用领域	说明
工业通信	Modbus、CAN等协议中用于帧校验
嵌入式系统	串口通信、无线模块、传感器数据校验
存储系统	U盘、Flash、SD 卡数据完整性校验
网络通信	以太网（CRC32）、Wi-Fi、PPP 协议
文件校验	ZIP、RAR、PNG、JPEG、Git 等使用 CRC 或更强的 Hash
数据传输	UART、SPI、I2C、USB 等通信中常用

注：C/C++中可以使用现成的CRC库如：crc16.h等

九、 Modbus CRC-16实现：

1. Modbus CRC-16是什么？

在 Modbus 协议 中，为了保证通信数据的完整性，每个通信帧（请求或响应）都会在数据部分的最后附加一个 2 字节（16 位）的 CRC 校验码，用于检测数据在传输过程中是否发生错误。

- Modbus 使用的 CRC 类型：CRC-16
- 多项式：0x8005（即 $x^{16} + x^{15} + x^2 + 1$ ）
- 初始值：0xFFFF
- 计算方式：按字节逐个处理
- 最终结果：CRC 低字节在前，高字节在后（即小端序 / Little Endian）

Modbus CRC-16 与标准 CRC-16-IBM（多项式 0x8005）的计算过程基本一样，但最终 CRC 值的字节顺序是颠倒的（低字节在前）！

2. Modbus CRC-16计算步骤：

1. 初始化

CRC 寄存器初始值为：0xFFFF

2. 对每一个字节进行 8 次处理（即一个字节一个字节来）

对于数据中的每一个字节（8 bit），步骤如下：

(1) 将当前 CRC 值与数据字节进行异或（XOR）

(2) 进行 8 次循环（对应 8 个 bit）：

判断 CRC 最低位（LSB）是否为 1：

如果是 1：右移一位与多项式 0xA001（即 0x8005 的位反转，也就是 $0x8005 \wedge 0xFFFF$ ）做异或

如果是 0：仅右移一位（这等价于： $CRC = (CRC \gg 1) \wedge polynomial$ ，当最低位为 1 时）

处理完 8 次后，处理下一个字节

(3) 最终 CRC 值

- 处理完所有数据字节后，得到一个 16 位的 CRC 值
- 将这个 CRC 值的高低字节互换（小端序，Low Byte First）即：先发送低字节，再发送高字节
- 附加到通信帧的最后两个字节

3.多项式说明

项目	值/说明
标准多项式	0x8005（即 $x^{16} + x^{15} + x^2 + 1$ ）
Modbus实际使用的多项式(等效)	0xA001（即 0x8005 的 位反转，用于简化计算）
初始值	0xFFFF

项目	值/说明
最终CRC字节序	低字节在前，高字节在后（小端序）

可以直接使用多项式 **0xA001** 来简化运算逻辑，效果与 0x8005 等效（通过位反转和异或初值实现）。

4. Modbus CRC-16 计算示例(简略)

假设我们要计算的数据是：**0x01 0x03 0x00 0x00 0x00 0x01**（这是一个典型的 Modbus 读取保持寄存器的请求帧，功能码 0x03，读取 1 个寄存器）

经过 CRC-16 计算后，得到的正确 CRC 值是：**0xC5F0**

但注意：在 Modbus 帧中，你要将这个 CRC 值按 低字节在前发送，即：

- **CRC 低字节：0xF0**
- **CRC 高字节：0xC5**
- **所以实际附加到帧尾的是：0xF0 0xC5**

十、Modbus CRC-16代码实现

```
#include <stdint.h>
#include <stdio.h>

uint16_t modbus_crc16(const uint8_t *data, uint16_t length)
{
    uint16_t crc = 0xFFFF;
    for (uint16_t i = 0; i < length; i++)
    {
        crc ^= data[i];
        for (uint8_t j = 0; j < 8; j++)
        {
            if (crc & 0x0001)
            {
                crc >>= 1;
                crc ^= 0xA001;
            }
            else
            {
                crc >>= 1;
            }
        }
    }
    return crc;
}

int main()
{
    uint8_t data[] = {0x01, 0x03, 0x00, 0x00, 0x00, 0x01};
    uint16_t crc = modbus_crc16(data, sizeof(data));
    printf("CRC-16: 0x%04X\n", crc);           // 应该输出 0xC5F0
}
```

```
    printf("CRC Bytes: 0x%02X 0x%02X\n", crc & 0xFF, (crc >> 8) & 0xFF); // 0xF0
    0xC5
    return 0;
}
```

注意：

虽然计算结果是 0xC5F0，但发送时请按 **低字节在前**，即：**0xF0 0xC5**